19. ArrayList — Quick Notes

Definition

- Implementation of `List` interface
- Dynamic Sized Array
- Automatically grows when full

ArrayList = Dynamic Array

Why ArrayList?

Problem with Normal Array

int arr[] = new int[3];

If array becomes full:

- Cannot add more elements
- Fixed size

ArrayList Solution

ArrayList<Integer> list = new ArrayList<>();

When full:

- Resizes automatically
- No manual memory management

Advantages of ArrayList

1. Dynamic Size

- Grows automatically
- No fixed size limitation

2. Rich Library Methods

Methods available:

- add()
- remove()
- contains()
- indexOf()
- get()
- set()

3. Easy to Use

No need to:

- create bigger array
- copy elements manually

Real World Example

Suppose: Online Course Registration
Unknown number of students.

❌ Fixed Array
Student arr[] = new Student[2000];
May waste memory.
✅ ArrayList
ArrayList<Student> students = new ArrayList<>();
Grows only when needed.

Creating ArrayList

1. Empty ArrayList

ArrayList<Integer> list = new ArrayList<>();
Initially:[]
Most common approach.

Adding Elements

list.add(10);
list.add(20);
list.add(30);

System.out.println(list);

Output: [10, 20, 30]

Important

add(): adds element at END by default.

Non-Primitive Types Only

❌ **Invalid:** ArrayList<int> list;
✅ **Valid:** ArrayList<Integer> list;

Use Wrapper Classes:

Primitive	Wrapper
int	Integer
char	Character
double	Double
boolean	Boolean

---

2. ArrayList with Capacity

ArrayList<Integer> list = new ArrayList<>(10);

Meaning

Initial Capacity: 10
Java pre-allocates memory.

Important

Capacity ≠ Size
ArrayList<Integer> list = new ArrayList<>(10);

Current: Size = 0, Capacity = 10

---

3. Create from Another Collection

```
HashSet<Integer> set = new HashSet<>();
set.add(10);
set.add(20);

ArrayList<Integer> list = new ArrayList<>(set);
```

Output: [10, 20]

Important
Can create ArrayList from:

- HashSet
- TreeSet
- LinkedList
- Any Collection

ArrayList Syntax

```
import java.util.ArrayList;
ArrayList<Integer> list = new ArrayList<>();
```

Common Methods

Method	Purpose
add(e)	Add element
get(i)	Get element
set(i,e)	Replace element
remove(i)	Remove element
contains(e)	Check existence
indexOf(e)	First occurrence
size()	Number of elements

Example

```
ArrayList<Integer> list = new ArrayList<>();
list.add(10);
list.add(20);

System.out.println(list.contains(20));
```

Output: true

Important Quick Points

Topic	Point
ArrayList	Dynamic Sized Array
Interface	Implements List
Duplicates	Allowed
Ordering	Preserved
Access	Index Based
add()	Adds at end
Primitive Types	Not allowed
Capacity	Internal storage size
Size	Actual elements count
Most Used List	ArrayList